

Your RAG vs Industry-Level RAG

This is the differentiation list. Each gap shows what you have, what industry has, and what you need to learn to bridge it.

1. Query Processing

You have:

```
python

user_query = input("\nUser:\n").strip()
run_query(user_query) # Passes raw query directly to retrieval
```

What this means: Takes the user's exact question, embeds it, searches. Fast but narrow — only catches exact semantic matches.

Industry has:

1. **Query expansion** — "What's machine learning?" becomes 3-4 sub-queries:
 - "Define machine learning"
 - "What are the types of machine learning"
 - "History and applications of ML"
 - Each retrieves separately, results merged
2. **HyDE (Hypothetical Document Embedding)** — Asks the LLM to generate a fake "ideal answer", then uses *that embedding* to search instead of the raw query
 - Catches relevant docs the query alone wouldn't
3. **Query classification** — "Is this factual, opinion, summarization, or comparison?"
 - Routes to different retriever weights or strategies
4. **Query rewriting** — Fixes spelling, expands abbreviations, clarifies pronouns
 - "ML" → "machine learning", "it" → "the algorithm"

What you need to learn:

- Prompt engineering (how to make the LLM generate good hypothetical answers)
- Multi-query retrieval strategies
- When to spend compute on query understanding vs hitting retrieval fast

Effort to implement:

Easy. Takes 20 lines of code. Start with HyDE: have the LLM generate 1-2 plausible answers, embed them, search with those vectors alongside the original query.

2. Evaluation & Metrics

You have:

- Print statements in the code
- LangSmith traces (logs what happened, but no scoring)

What this means: You can see the pipeline executed, but not *if it was correct*.

Industry has:

Automated scoring on every query:

1. **Faithfulness** — Did the answer come from the context, or did it hallucinate?
 - Scored 0-1 using an LLM (ask it: "Can this answer be inferred from the context?")
2. **Answer relevance** — Does the answer actually address the question?
 - Scored 0-1
3. **Context precision** — Of the chunks retrieved, what % were actually used?
 - If you retrieved 3 chunks but only 1 was relevant, score = 0.33
4. **Context recall** — Of all chunks that *could* answer the question, what % did you retrieve?
 - Hard to measure without ground truth, but approximated
5. **Latency & token usage** — Cost and speed per query
6. **User feedback loop** — "Was this answer helpful?" signals feed back into training signal

Aggregated metrics:

- P50/P95 latency
- Hallucination rate per document
- Answer quality per question type
- Cost per successful query

Dashboards:

- Real-time: what % of queries had zero hallucination in the last hour?
- Historical: is answer quality degrading over time?

What you need to learn:

- RAGAS framework (or DeepEval) — the tooling that scores this automatically
- How LLMs score subjective things like "faithfulness"
- Ground truth creation — what makes a "correct answer"?
- Monitoring & alerting — when to page someone if quality drops

Effort to implement:

Medium. RAGAS is a library. Basic setup = 50 lines, but running evals on 100 queries takes 5-10 minutes (evals use LLM calls). You need infrastructure to collect results and visualize them.

3. Error Handling & Guardrails

You have:

```
python

if not user_query:
    continue

run_query(user_query) # No validation, no output checks
```

What this means: Passes anything through. If LLM hallucinates, you return it as-is.

Industry has:

Input guards:

1. **Prompt injection detection** — "Forget your instructions and..." → blocked
2. **PII detection** — "My credit card is 1234-5678..." → anonymize or reject
3. **Off-topic routing** — "Help me hack into X" → refuse or escalate
4. **Rate limiting** — Protect against DoS

Output guards:

1. **Hallucination detector** — "The answer mentions facts not in the context" → flag or filter
2. **Refusal detector** — "I can't answer that" from the LLM → pass through, don't pretend
3. **Length limiter** — Ensure response doesn't exceed contract
4. **Toxicity filter** — Just in case

Soft guardrails (policy, not hard blocks):

- Log risky queries for manual review

- Tag uncertain answers
- Route low-confidence queries to a human

What you need to learn:

- Guardrails AI framework (or NeMo Guardrails)
- How to write safety classifiers
- The tradeoff between false positives (rejecting good queries) and false negatives (letting bad queries through)

Effort to implement:

Medium. Basic input validation = 50 lines. Full guardrails framework = 200 lines + design decisions.

4. Retrieval Caching

You have:

```
python

retriever = ContextualCompressionRetriever(
    base_compressor=reranker,
    base_retriever=ensemble_retriever,
)
# Runs full retrieval every time
```

What this means: Every query hits the database and embeddings model, even if it's similar to a previous query.

Industry has:

Semantic caching:

1. Hash the query embedding
2. Check cache: "Is there a similar query we've already answered?"
3. If yes: return cached answer (skip retrieval + LLM)
4. If no: run full pipeline, cache the result

Example:

- User A: "What is backpropagation?" → 5 seconds, then cached
- User B (1 minute later): "Explain backprop" → embedding is 0.98 similar, return cache instantly

Cost/latency savings:

- 60-80% of queries in real usage are similar to previous ones
- Cache hit = save: embedding call, retrieval call, LLM call = ~4-8 seconds + \$0.02-0.10

What you need to learn:

- Semantic similarity thresholds (when is 0.95 similarity "close enough"?)
- Cache invalidation (when to drop old results)
- TTL strategies (how long to keep a cached answer?)
- Redis or in-memory caching

Effort to implement:

Easy. GPTCache or similar is a drop-in wrapper. 30 lines of code.

5. Deployment & API Layer

You have:

```
python

if __name__ == "__main__":
    while True:
        user_query = input("\nUser:\n").strip()
        run_query(user_query)
```

What this means: Terminal loop. Single-threaded, synchronous, blocks on each query.

Industry has:

FastAPI/LangServe endpoint:

```
python

@app.post("/query")
async def query_endpoint(request: QueryRequest) -> QueryResponse:
    # Async: handle 100 users simultaneously
    # Timeout: queries that take >30s fail instead of hang
    # Retry logic: if embedding API fails, retry exponentially
    # Circuit breaker: if retrieval service is down, fail fast
    result = await run_query_with_timeout(request.question, timeout=30)
    return QueryResponse(answer=result, timestamp=now(), trace_id=...)
```

Production additions:

1. **Async handlers** — Serve 100 queries simultaneously
2. **Request validation** — OpenAPI schema, type checking
3. **Rate limiting** — Max 100 queries/hour per user
4. **Request/response logging** — Every interaction recorded for debugging
5. **Tracing headers** — Trace a query through the system
6. **Health checks** — `/health` endpoint for load balancers
7. **Graceful shutdown** — Finish inflight requests before exiting

What you need to learn:

- FastAPI or Flask
- Async Python (async/await)
- Deployment (Docker, Kubernetes, or serverless)
- Error codes and HTTP semantics

Effort to implement:

Medium-Hard. Basic FastAPI wrapper = 100 lines. Production-grade = 500+ lines (logging, monitoring, error handling).

6. Incremental Indexing

You have:

```
python

def create_vector_store(chunks: list, ...):
    # Re-embeds ALL chunks every time
    for i in range(0, len(chunks), BATCH_SIZE):
        batch = chunks[i : i + BATCH_SIZE]
        vector_store.add_documents(batch)
```

What this means: If you have 10,000 documents and add 5 new ones, you re-embed all 10,000 + 5.

Industry has:

Hash-based change detection:

```
python
```

```
def incremental_index(new_pdfs: list, existing_index: VectorStore):
    for pdf in new_pdfs:
        hash_new = hash_file(pdf)
        hash_old = existing_index.metadata(pdf_name)

        if hash_new == hash_old:
            continue # No change, skip

        chunks = split_pdf(pdf)
        vectors = embed(chunks) # Only embed the changed PDF
        existing_index.add_documents(chunks) # Append, don't rebuild
```

Cost/latency savings:

- Initial index: 10,000 docs, 5 minutes
- Update with 5 new docs: 10 seconds (not 5 minutes)

What you need to learn:

- File hashing and change detection
- Chroma collection upsert (update-or-insert) vs add
- Versioning strategies (when to fully rebuild vs append)

Effort to implement:

Easy. 50 lines. Biggest decision is *when* to do a full rebuild (monthly? quarterly?).

7. Metadata Filtering & Multi-Collection

You have:

```
python

vector_store = Chroma(
    persist_directory="chromaDb",
    embedding_function=embedding_model,
)
# All documents in one collection, no filtering
```

What this means: A query searches *all* documents. If you're building a Q&A for multiple customers, each customer can see each other's data.

Industry has:

Tenant isolation:

```
python

# Each customer has their own Chroma collection
customer_1_store = Chroma(
    name="customer_1_docs",
    embedding_function=embedding_model,
    persist_directory="chromaDb",
)

customer_2_store = Chroma(
    name="customer_2_docs",
    embedding_function=embedding_model,
    persist_directory="chromaDb",
)

# Retriever only searches the right collection
vector_retriever = customer_1_store.as_retriever()
```

Or metadata-based filtering:

```
python

# Single collection, but filter by metadata
retriever = vector_store.as_retriever(
    search_kwargs={
        "filter": {
            "customer_id": user_id,
            "document_type": "policy", # Only search policies
            "date_created": {"$gte": "2024-01-01"} # Only recent docs
        }
    }
)
```

Benefits:

- Security: Users only see their data
- Precision: Reduce search space (fewer irrelevant chunks)
- Cost: Fewer vectors to search through

What you need to learn:

- Chroma metadata filters and their syntax
- How to attach customer_id, doc_type, etc. to chunks at indexing time
- Multi-collection architectures

Effort to implement:

Easy. Modify the chunking function to add metadata, then pass `filter` to retriever. 20 lines.

8. Observability & Monitoring

You have:

```
python

@traceable(name="rag_query", metadata={...})
def run_query(question: str) -> str:
    # LangSmith logs what happened
```

What this means: You can see traces in LangSmith, but no dashboards, no alerts.

Industry has:

Real-time metrics:

1. **Latency percentiles** — P50, P95, P99 per component
 - Is retrieval taking 2s or 5s today? (degradation detection)
2. **Query volume** — How many queries/hour?
 - Spike detection (double the normal volume = something's wrong)
3. **Error rates** — What % of queries error?
 - Retrieval service down? Embedding API timing out?
4. **Token usage & costs** — \$/query, tokens/query
 - Cost trending, budget forecasting
5. **Answer quality** (from evals) — Hallucination rate, answer relevance
 - Trending over time: is the system getting worse?
6. **User satisfaction** — Click-through, feedback, ratings
 - Users marking answers as "not helpful" = signal

Dashboard:

- Main page: green/yellow/red status of core metrics
- Drill-down: latency by retrieval type, by document type, by time of day
- Alerts: "Hallucination rate exceeded 5%, paging on-call engineer"

Logging:

- Every query: input, retrieved docs, LLM response, latency, cost

- Sent to: logs (CloudWatch, ELK), metrics (Prometheus), traces (LangSmith)
- Retention: keep for 30 days for analysis, 1 year for auditing

What you need to learn:

- Prometheus or similar (metrics collection)
- Grafana or similar (dashboarding)
- Alert rules and escalation policies
- Structured logging (JSON format, not print statements)

Effort to implement:

Medium-Hard. Basic Prometheus + Grafana = 200 lines config + dashboard JSON. Full setup with alerting = 1000+ lines.

9. Versioning & A/B Testing

You have:

```
python

llm = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash", # Fixed model, no versioning
)
```

What this means: If you want to test a new embedding model or LLM, you have to swap the code and re-run everything.

Industry has:

Model registry:

```
python
```

```

EMBEDDING_MODEL_VERSIONS = {
    "v1": {"name": "text-embedding-004", "production": False},
    "v2": {"name": "text-embedding-004-latest", "production": True},
}

LLM_VERSIONS = {
    "v1": {"name": "gemini-2.5-flash", "temperature": 0.2, "production": True},
    "v2": {"name": "gemini-2.0-pro", "temperature": 0.1, "production": False},
}

def get_retriever(user_id: str):
    # 90% of users get v1, 10% get v2
    if hash(user_id) % 100 < 10:
        return build_retriever(EMBEDDING_MODEL_VERSIONS["v2"])
    else:
        return build_retriever(EMBEDDING_MODEL_VERSIONS["v1"])

```

Benefits:

- Gradual rollout: test new models on 1% of traffic first
- Comparison: "v2 is 0.3s slower but 5% more accurate"
- Rollback: if new version breaks, flip a flag back to old version
- Experimentation: compare retrieval strategies A/B without rebuilding index

What you need to learn:

- Feature flags and configuration management
- Canary deployments (5% → 10% → 50% → 100% traffic)
- Statistical significance testing (did v2 actually improve things?)

Effort to implement:

Medium. Basic flag system = 50 lines. With analytics = 200+ lines.

10. Scaling & Optimization

You have:

```

python

# ChromaDB runs in-memory or SQLite on disk
# Embedding done in single-threaded loop
# Everything on one machine

```

What this means: Works fine for 10-100 queries/day. Breaks at 1000+/day.

Industry has:

Distributed architecture:

```
User Request
  ↓
Load Balancer (serves 100 replicas of the API server)
  ↓ (each replica)
Retrieval Service (separate machine, scales independently)
  ↓
ChromaDB or Elasticsearch (distributed vector DB, sharded across 10 machines)
  ↓
Embedding Service (batch + cache, 50 GPU machines)
  ↓
LLM Inference (batched, GPU cluster or API)
```

Optimization techniques:

1. **Batch retrieval** — Process 10 queries' embeddings in parallel
2. **GPU optimization** — Use GPUs for embedding & LLM inference
3. **Quantization** — Store embeddings in lower precision (768D → 128D), 6× smaller
4. **Caching** — Cache popular embeddings, retrieval results
5. **Index optimization** — Use HNSW (Hierarchical Navigable Small World) instead of brute force

Cost/latency:

- Current: 100 queries/day, 1 GPU machine, \$50/month
- Industry: 100,000 queries/day, 50 machines, \$5000/month (but \$0.05/query)

What you need to learn:

- Kubernetes or serverless (horizontal scaling)
- GPU programming (optional, but helps)
- Distributed tracing (OpenTelemetry)
- Cost optimization (spot instances, batch processing)

Effort to implement:

Hard. This is DevOps/SRE work. Start simple (single machine, but async), then scale as traffic grows.

Quick Mapping: What to Learn When

Phase 1 (Next 1-2 weeks) — Foundation

These are **highest ROI per hour spent**:

1. **Query expansion + HyDE** (2 hours)
 - Improves retrieval quality immediately
 - 20 lines of code
2. **RAGAS evaluation** (3 hours)
 - Gives you visibility into what's breaking
 - Can't improve what you don't measure
3. **Basic guardrails** (2 hours)
 - Blocks obvious bad inputs
 - 50 lines of code
4. **Semantic caching** (1 hour)
 - Cheap latency/cost win
 - GPTRCache is a wrapper

Total effort: 8 hours. Impact: 2-3x better answers, visibility into quality, faster responses.

Phase 2 (Weeks 3-4) — Robustness

1. **FastAPI wrapper** (4 hours)
 - Makes it deployable
 - Required before production
2. **Metadata filtering** (2 hours)
 - Multi-tenant support
 - Precision improvement
3. **Incremental indexing** (2 hours)
 - Saves time on updates

Total: 8 hours. Impact: Deployable, scalable, maintainable.

Phase 3 (Month 2+) — Polish

1. **Observability stack** (20 hours)

- Prometheus + Grafana
 - Structure logging
2. **Scaling** (40+ hours)
 - Kubernetes, distributed retrieval
 - GPU optimization
 3. **A/B testing** (10 hours)
 - Experiment framework

Total: 70+ hours. Impact: Production-grade reliability, cost optimization, data-driven improvements.

The Honest Take

Your current code (Level 3) is **genuinely good** for a prototype. It has the hard parts right: hybrid retrieval, reranking, tracing.

The gaps are not "missing features" — they're **operational maturity**.

Think of it like building a car:

- **You have:** Engine, transmission, wheels (works, can drive)
- **Industry has:** Engine, transmission, wheels + airbags, crumple zones, ABS, cruise control, diagnostics (safe, efficient, reliable)

Both can get you from A to B. One is safe to give to 100,000 people; one isn't.

Your learning path should be:

1. Measure (evals) — Can't improve without knowing what's broken
2. Experiment (guardrails + query expansion) — Small changes, big impact
3. Deploy (FastAPI) — Make it usable by others
4. Scale (caching, async, monitoring) — Handle real load
5. Optimize (A/B testing, advanced retrieval) — Squeeze performance

Start with #1 and #2 *this week*. You'll immediately see:

- "My hallucination rate dropped 20%"
- "Query expansion finds answers naive search missed"

That feedback loop (measure → experiment → see improvement) is what keeps you motivated and focused.